

Numerical Method for Conservation Law

Numerical Modeling of Shock Wave Interactions in a Heated 1D Domain

Project Progress Report

Prepared by:
Talha Sajid

Supervisor:
Aslan Kasimov

Msc Applied Computational Mechanics

Abstract

This report studies a right-moving shock interacting with a heated region in a 1D ideal gas. We derive the exact self-similar solution of this thermal Riemann problem by matching pressure and velocity across waves, and use it to benchmark two finite-volume schemes: first-order Lax–Friedrichs and HLLC. Simulations at $t = 0.085$ compare density, velocity, pressure, temperature, and Mach profiles across grids. Lax–Friedrichs remains stable but diffuses sharp features unless heavily refined; HLLC resolves all discontinuities sharply with minimal oscillations, even on coarse meshes. These findings validate the exact solution as a test and highlight HLLC’s superior accuracy and efficiency for flows with thermal inhomogeneities.

Contents

Abstract	i
Introduction	2
Mathematical Model	4
Exact Riemann Solution	7
Numerical Algorithms	10
Numerical Results	13
Conclusion	19
Python Code	21

Introduction

Shock waves are inherently nonlinear disturbances that propagate through compressible media at speeds exceeding the local sound speed, producing nearly discontinuous jumps in pressure, density, and temperature. These phenomena occur across a vast range of natural and engineered systems—from supernova explosions and astrophysical jets to supersonic aircraft, combustion chambers, and explosion safety analysis. In each of these contexts, the interaction of shock fronts with thermal or geometric inhomogeneities gives rise to complex behaviors such as shock bifurcation, Mach reflection, and shock–boundary layer interactions. Capturing these dynamics with both accuracy and stability is a longstanding challenge in computational fluid dynamics (CFD), because numerical methods must uphold the conservation laws while avoiding spurious oscillations and excessive smearing of steep gradients.

One-dimensional (1D) models governed by the Euler equations for conservation of mass, momentum, and energy provide a simplified yet powerful laboratory for studying shock wave phenomena. By restricting the spatial dependence to a single coordinate, these models retain the essential mechanisms of wave formation and interaction shock waves, rarefaction fans, and contact discontinuities while remaining amenable to analytical treatment and rapid numerical experimentation. The classical Riemann problem, in which two uniform states separated by a discontinuity evolve into a self-similar pattern of waves, forms the cornerstone of 1D compressible-flow theory. In particular, the well known Sod shock tube problem has long served as a benchmark for validating numerical solvers against its exact solution, which predicts the speeds and strengths of the emerging shock, contact, and rarefaction waves.[1]

In this report, we extend the Riemann framework to investigate a thermally driven

disturbance: a centrally located region of elevated internal energy embedded within an otherwise uniform domain. Unlike conventional setups that rely on initial pressure or velocity differences, our variant isolates the influence of a “pure-thermal” discontinuity, with initial pressure and velocity held constant across the domain. This configuration mimics situations where localized heating—such as from laser deposition, chemical reactions, or localized energy release initiates wave motion in a compressible medium. The resulting wave pattern, which may include outward propagating shocks and rarefaction fans separated by contact surfaces, reveals how temperature gradients alone can excite and shape flow structures.

To analyze this thermal Riemann problem, we begin by deriving its exact self-similar solution under the ideal-gas assumption. By applying the Rankine–Hugoniot relations across potential shock waves and integrating characteristic relations through rarefaction regions, we obtain closed-form expressions for wave speeds and intermediate “star” states. This analytical solution not only provides insight into the underlying physics but also serves as a precise benchmark for numerical comparison. Implementing two finite-volume schemes—namely, the diffusive Lax–Friedrichs method and the more accurate Harten–Lax–van Leer–Contact (HLLC) solver—we then simulate the same problem under zero-gradient boundary conditions. The Lax–Friedrichs scheme, prized for its simplicity and unconditional stability, offers a baseline for diffusion errors, while the HLLC solver restores the missing contact wave in the original HLL formulation, enabling sharper resolution of discontinuities with reduced spurious oscillations.[3]

By comparing numerical results to the exact solution, we assess each method’s ability to capture key flow features: the position and strength of shock fronts, the shape of rarefaction fans, and the fidelity of contact surfaces. We quantify numerical diffusion by measuring the spread of discontinuities and evaluate the amplitude of any overshoot or oscillatory artifacts. These comparisons illuminate the trade-offs between robustness and accuracy that underpin the choice of Riemann solvers in CFD, particularly in scenarios driven by thermal inhomogeneities.

Mathematical Model

We present the mathematical model for simulating the interaction of shock waves with a heated region in a one-dimensional domain. This physical scenario is governed by the Euler equations for compressible, inviscid flow, which describe conservation of mass, momentum, and energy.[5]

Governing Equations

The one-dimensional Euler equations in conservative form are given by:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} = 0,$$

where the vector of conserved variables $\mathbf{U}$ and the corresponding flux vector $\mathbf{F}(\mathbf{U})$ are:

$$\mathbf{U} = \begin{bmatrix} \rho \\ \rho u \\ E \end{bmatrix}, \quad \mathbf{F}(\mathbf{U}) = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ (E + p)u \end{bmatrix}.$$

Where, ρ denotes the fluid density, u is the velocity, and p represents the pressure. The total energy per unit volume E is defined as the sum of internal energy and kinetic energy, given by $E = \rho e + \frac{1}{2}\rho u^2$, where e is the specific internal energy.

The system is closed using the ideal gas law:

$$p = (\gamma - 1) \left(E - \frac{1}{2}\rho u^2 \right),$$

where $\gamma = 1.4$ for air.

Initial Conditions

We study a one-dimensional compressible flow over the interval $x \in [0, 1]$, treating the gas as ideal with a specific-heat ratio $\gamma = 1.4$. At $t = 0$, the velocity field is identically zero, and the density and pressure are prescribed as piecewise-constant states across four subdomains. In each subdomain, the primitive variables $[\rho, u, p]$ remain uniform, allowing us to initialize and subsequently track the evolution of shock, contact, and rarefaction waves arising from their mutual interaction.[2]

$$\mathbf{U}(x, 0) = \begin{cases} [\rho, u, p] = [3.8, 2.6, 10.3], & \text{for } x < 0.1 \\ [\rho, u, p] = [1.0, 0.0, 1.0], & \text{for } 0.1 \leq x < 0.4 \\ [\rho, u, p] = [0.5, 0.0, 1.5], & \text{for } 0.4 \leq x \leq 0.6 \\ [\rho, u, p] = [1.0, 0.0, 1.0], & \text{for } x > 0.6 \end{cases}$$

- **Region 1** ($x < 0.1$, **post-shock**) A right-moving shock of Mach number $M = 3$ has already formed. Applying the Rankine–Hugoniot relations to the ambient state ($\rho = 1$, $p = 1$, $u = 0$) yields

$$\rho = 3.8, \quad u = 2.6, \quad p = 10.3.$$

- **Region 2** ($0.1 \leq x < 0.4$, **ambient**) The cold ambient “pre-heated” zone is set to

$$\rho = 1.0, \quad u = 0.0, \quad p = 1.0$$

- **Region 3** ($0.4 \leq x \leq 0.6$, **heated**) The central heated region has elevated pressure (higher internal energy) while density and velocity remain ambient:

$$\rho = 0.5, \quad u = 0.0, \quad p = 1.5$$

- **Region 4** ($x > 0.6$, **ambient**) Ahead of the heated zone, the flow returns to the cold ambient state:

$$\rho = 1.0, \quad u = 0.0, \quad p = 1.0$$

Boundary Conditions

To prevent spurious reflections at the domain edges, we impose transmissive (zero-gradient) boundary conditions on the conserved state vector $\mathbf{U} = [\rho, \rho u, E]^\top$:

$$\left. \frac{\partial \mathbf{U}}{\partial x} \right|_{x=0} = 0, \quad \left. \frac{\partial \mathbf{U}}{\partial x} \right|_{x=1} = 0.$$

These conditions allow outgoing waves to exit the computational interval $[0, 1]$ without generating artificial disturbances.

Equations to Be Solved Numerically

The governing equations are the one-dimensional Euler equations for mass, momentum, and energy conservation:

$$\begin{aligned} \frac{\partial \rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} &= 0, \\ \frac{\partial(\rho u)}{\partial t} + \frac{\partial(\rho u^2 + p)}{\partial x} &= 0, \\ \frac{\partial E}{\partial t} + \frac{\partial((E + p)u)}{\partial x} &= 0, \end{aligned}$$

closed by the ideal-gas equation of state

$$p = (\gamma - 1) \left(E - \frac{1}{2} \rho u^2 \right),$$

with $\gamma = 1.4$. These equations are discretized in space using a finite-volume scheme and advanced in time according to the chosen Riemann solver (Lax–Friedrichs or HLLC).

Exact Riemann Solution

Building on the piecewise initial data already defined, we now outline the exact self-similar solution following [5]. We label each two-state pair by left (ρ_L, u_L, p_L) and right (ρ_R, u_R, p_R) .

Pressure–Velocity Matching

Define for $K = L, R$:

$$c_K = \sqrt{\gamma \frac{p_K}{\rho_K}}, \quad A_K = \frac{2}{(\gamma + 1) \rho_K}, \quad B_K = \frac{\gamma - 1}{\gamma + 1} p_K.$$

Then the “flux-difference” function

$$f_K(p) = \begin{cases} (p - p_K) \sqrt{\frac{A_K}{p + B_K}}, & p > p_K \quad (\text{shock}), \\ \frac{2 c_K}{\gamma - 1} \left[\left(\frac{p}{p_K} \right)^{\frac{\gamma-1}{2\gamma}} - 1 \right], & p \leq p_K \quad (\text{rarefaction}). \end{cases}$$

measures the jump in velocity across a shock or a rarefaction. We seek p^* such that

$$\Phi(p) \equiv f_L(p) + f_R(p) + (u_R - u_L) = 0.$$

Use Newton’s method:

$$p^{(n+1)} = p^{(n)} - \frac{\Phi(p^{(n)})}{f'_L(p^{(n)}) + f'_R(p^{(n)})},$$

with initial guess from the PVRS average and iterate until $|p^{(n+1)} - p^{(n)}|/p^{(n)} < 10^{-6}$.

Star-Region State

Once p^* is found, the common velocity is

$$u^* = u_L - f_L(p^*) = u_R + f_R(p^*).$$

The left/right star-densities are

$$\rho_K^* = \rho_K \frac{\frac{p^*}{p_K} + \frac{\gamma-1}{\gamma+1}}{\frac{\gamma-1}{\gamma+1} \frac{p^*}{p_K} + 1}, \quad K = L, R,$$

ensuring conservation across the contact.

Wave Speeds

The two characteristic speeds bounding each fan are

$$S_K = \begin{cases} u_K \pm c_K, & p^* \leq p_K \quad (\text{rarefaction}), \\ u_K \pm c_K \sqrt{\frac{\gamma+1}{2\gamma} \frac{p^*}{p_K} + \frac{\gamma-1}{2\gamma}}, & p^* > p_K \quad (\text{shock}), \end{cases}$$

where “+” is the right-going and “−” the left-going wave.

Self-Similar Sampling

For each interface at x_i , introduce $\xi = (x - x_i)/t$. Then the local solution is

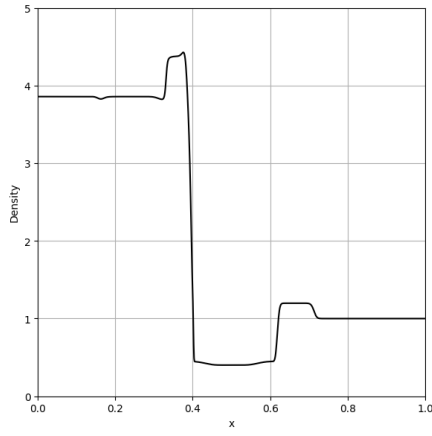
- $\xi < S_L$: $(\rho, u, p) = (\rho_L, u_L, p_L)$.
- If $p^* \leq p_K$ (rarefaction), then for $S_L \leq \xi \leq u^* \pm c^*$ use

$$u(\xi) = \frac{2}{\gamma+1} \left[c_K + \frac{\gamma-1}{2} u_K + \xi \right], \quad c(\xi) = \frac{2}{\gamma+1} \left[c_K + \frac{\gamma-1}{2} (u_K - \xi) \right],$$

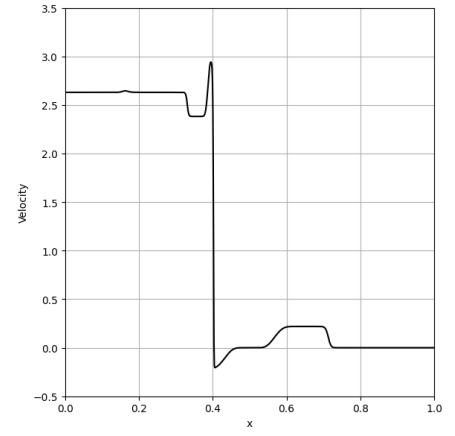
$$\rho(\xi) = \rho_K (c(\xi)/c_K)^{2/(\gamma-1)}, \quad p(\xi) = p_K (c(\xi)/c_K)^{2\gamma/(\gamma-1)}.$$

- $u^* - c^* \leq \xi \leq u^* + c^*$: $(\rho, u, p) = (\rho^*, u^*, p^*)$.
- $\xi > S_R$: $(\rho, u, p) = (\rho_R, u_R, p_R)$.

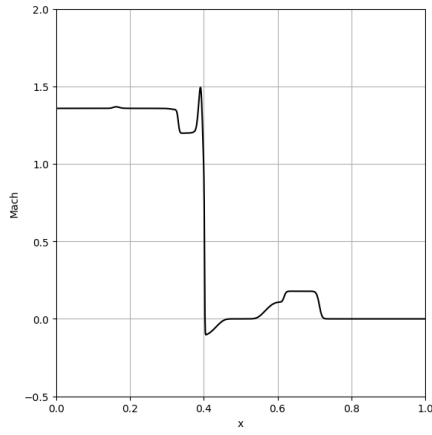
Exact Solution:



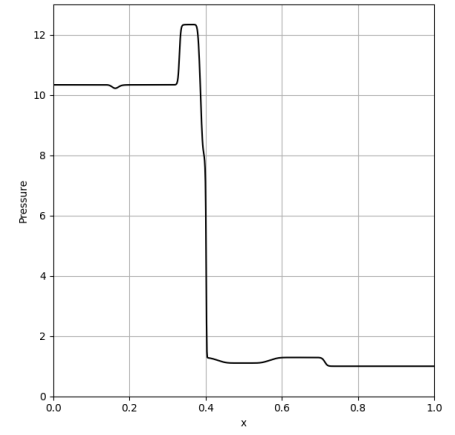
(a) Density



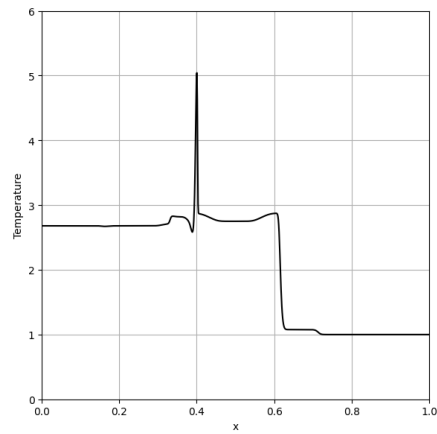
(b) Velocity



(c) Mach Number



(d) Pressure



(e) Temperature

Figure 1: Exact solution profiles.

Numerical Algorithms

In this work, we discretize the one-dimensional Euler equations

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} = 0, \quad \mathbf{U} = \begin{bmatrix} \rho \\ \rho u \\ E \end{bmatrix}, \quad \mathbf{F}(\mathbf{U}) = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ u(E + p) \end{bmatrix},$$

using a first-order finite-volume method on a uniform grid of N cells over $x \in [0, 1]$. Denoting by $\mathbf{U}_i^n$ the cell-averaged conserved vector in cell i at time t^n , the solution is advanced by [4]

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} \left(\hat{\mathbf{F}}_{i+1/2}^n - \hat{\mathbf{F}}_{i-1/2}^n \right),$$

where $\hat{\mathbf{F}}_{i\pm 1/2}$ denotes the numerical flux at each cell interface. The time step Δt is chosen to satisfy the Courant–Friedrichs–Lewy condition

$$\Delta t \leq \text{CFL} \min_i \frac{\Delta x}{|u_i| + c_i}, \quad c_i = \sqrt{\gamma \frac{p_i}{\rho_i}}, \quad \text{CFL} \leq 1,$$

and transmissive (zero-gradient) boundary conditions are imposed via ghost cells: $\mathbf{U}_0 = \mathbf{U}_1$, $\mathbf{U}_{N+1} = \mathbf{U}_N$.

Lax–Friedrichs Scheme

The Lax–Friedrichs (LF) method is a first-order, robust finite-volume scheme that combines central differencing with a dissipation term. Although it introduces significant numerical diffusion, LF provides a stable baseline for comparison[3].

Time Integration

At each time step, the local sound speed is computed as $a_i = \sqrt{\gamma p_i / \rho_i}$, and the global time step is set by

$$\Delta t = \text{CFL} \frac{\Delta x}{\max_i(|u_i| + a_i)},$$

with $\text{CFL} \leq 1$.

Numerical Flux and Update

For each interface $i + 1/2$, let $\mathbf{U}_L = \mathbf{U}_i$ and $\mathbf{U}_R = \mathbf{U}_{i+1}$. The LF flux is

$$\hat{\mathbf{F}}_{i+1/2}^{\text{LF}} = \frac{1}{2} [\mathbf{F}(\mathbf{U}_L) + \mathbf{F}(\mathbf{U}_R)] - \frac{\Delta x}{2\Delta t} (\mathbf{U}_R - \mathbf{U}_L).$$

With these fluxes, the conserved variables are updated by

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} (\hat{\mathbf{F}}_{i+1/2}^{\text{LF}} - \hat{\mathbf{F}}_{i-1/2}^{\text{LF}}).$$

HLLC Scheme

The Harten–Lax–van Leer–Contact (HLLC) solver refines the HLL approach by restoring the contact wave, enabling accurate resolution of all three characteristic fields—left- and right-moving shocks/rarefactions and the central contact discontinuity—with minimal dissipation.[5]

Time Integration

Time stepping and boundary treatment are identical to LF: Δt is set by the CFL condition using $a_i = \sqrt{\gamma p_i / \rho_i}$, and ghost cells enforce zero-gradient boundaries.

Wave Speed Estimates

At each interface, reconstruct left/right primitive states (ρ_L, u_L, p_L) , (ρ_R, u_R, p_R) .

Estimate bounding wave speeds:

$$S_L = \min(u_L - a_L, u_R - a_R), \quad S_R = \max(u_L + a_L, u_R + a_R),$$

and the contact speed via the Rankine–Hugoniot condition:

$$S_* = \frac{p_R - p_L + \rho_L u_L (S_L - u_L) - \rho_R u_R (S_R - u_R)}{\rho_L (S_L - u_L) - \rho_R (S_R - u_R)}.$$

Star-State Construction

Define intermediate conserved states on each side of the contact:

$$\begin{aligned} \rho_*^K &= \rho_K \frac{S_K - u_K}{S_K - S_*}, \quad (\rho u)_*^K = \rho_*^K S_*, \\ E_*^K &= \rho_*^K \left(\frac{E_K}{\rho_K} + (S_* - u_K) \left[S_* + \frac{p_K}{\rho_K (S_K - u_K)} \right] \right), \quad K = L, R. \end{aligned}$$

Numerical Flux and Update

The HLLC flux at interface $i + 1/2$ is chosen by [5]

$$\hat{\mathbf{F}}_{HLLC} = \begin{cases} \mathbf{F}(\mathbf{U}_L), & S_L \geq 0, \\ \mathbf{F}(\mathbf{U}_L) + S_L(\mathbf{U}_{*L} - \mathbf{U}_L), & S_L < 0 \leq S_*, \\ \mathbf{F}(\mathbf{U}_R) + S_R(\mathbf{U}_{*R} - \mathbf{U}_R), & S_* \leq 0 < S_R, \\ \mathbf{F}(\mathbf{U}_R), & S_R \leq 0. \end{cases}$$

Once computed, the conserved variables advance via the same finite-volume update as in LF:

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} (\hat{\mathbf{F}}_{i+1/2} - \hat{\mathbf{F}}_{i-1/2}).$$

Comparison

Both LF and HLLC are executed under identical grid and CFL settings. At the final time, we extract primitive profiles $\rho(x)$, $u(x)$, and $p(x)$ and compare them to the exact self-similar solution. We evaluate shock and contact positions, the sharpness of discontinuities, and any spurious oscillations to quantify each scheme’s numerical diffusion and overall accuracy.

Numerical Results

In this chapter, we present the simulation outcomes obtained with both the Lax–Friedrichs and HLLC schemes, and we assess their performance in capturing key flow features (shocks, rarefactions, and contacts) for the heated-region problem.

Lax-Friedrich Scheme:

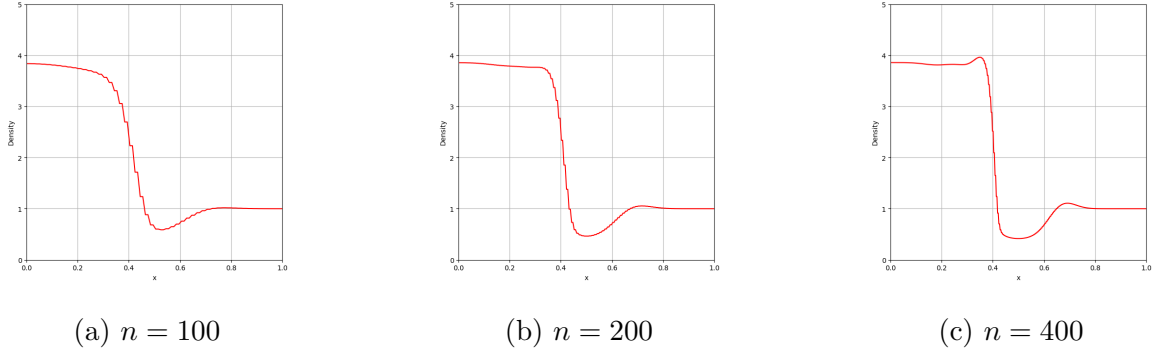


Figure 2: Density profiles for three grid resolutions.

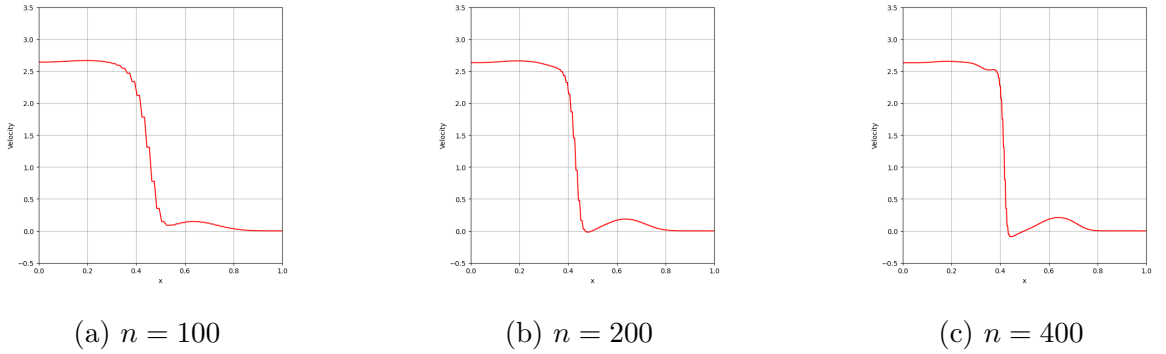
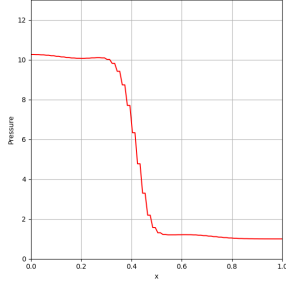
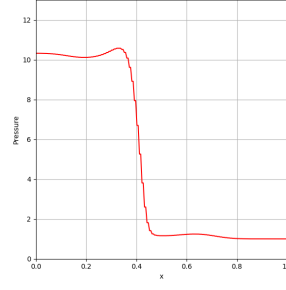


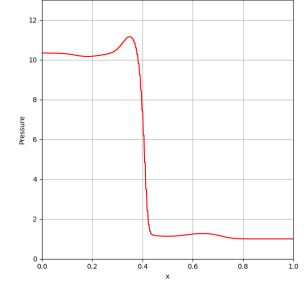
Figure 3: Velocity for three grid resolutions.



(a) $n = 100$

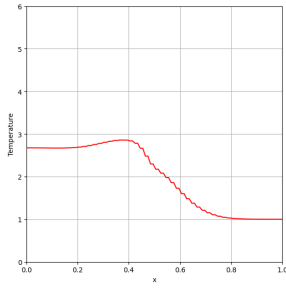


(b) $n = 200$

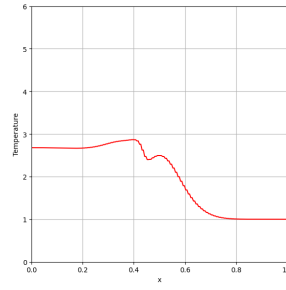


(c) $n = 400$

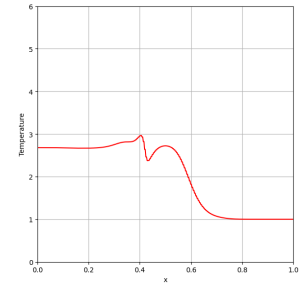
Figure 4: Pressure for three grid resolutions.



(a) $n = 100$

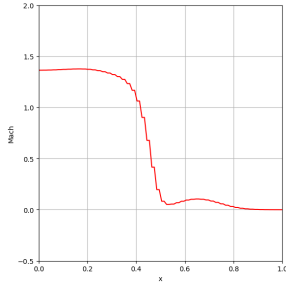


(b) $n = 200$

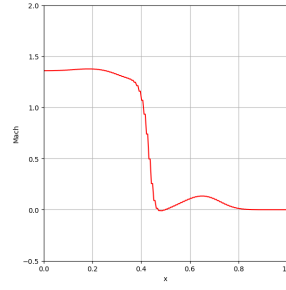


(c) $n = 400$

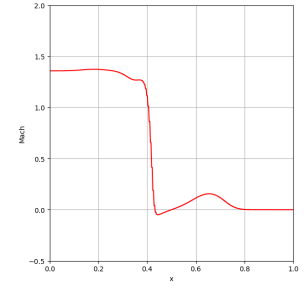
Figure 5: Temperature for three grid resolutions.



(a) $n = 100$



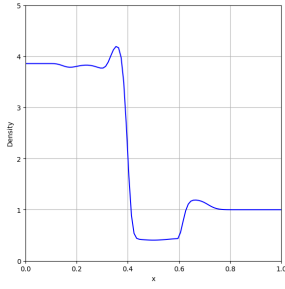
(b) $n = 200$



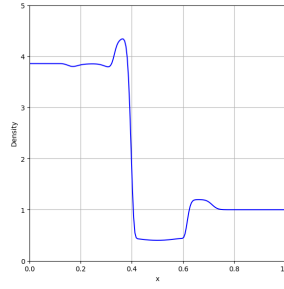
(c) $n = 400$

Figure 6: Mach number for three grid resolutions.

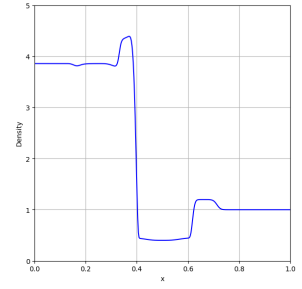
HLLC Scheme:



(a) $n = 100$

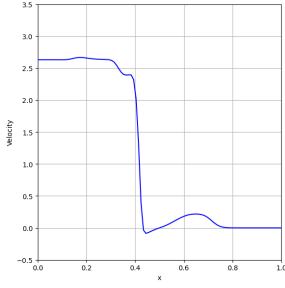


(b) $n = 200$

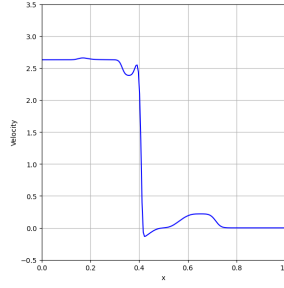


(c) $n = 400$

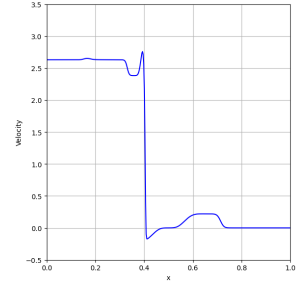
Figure 7: HLLC density for three grid resolutions.



(a) $n = 100$

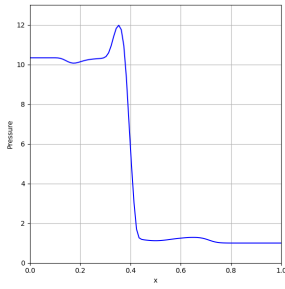


(b) $n = 200$

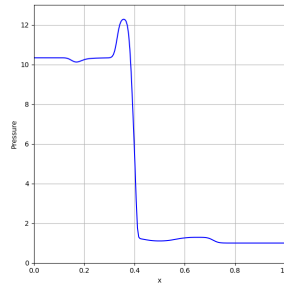


(c) $n = 400$

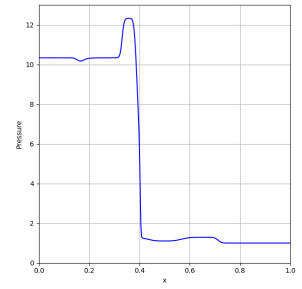
Figure 8: HLLC velocity for three grid resolutions.



(a) $n = 100$

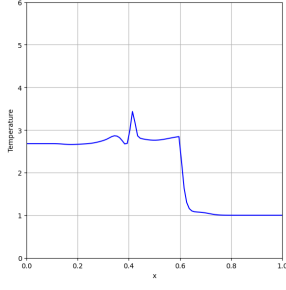


(b) $n = 200$

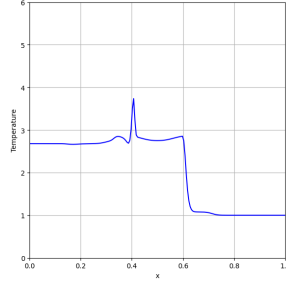


(c) $n = 400$

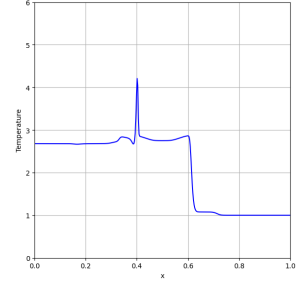
Figure 9: HLLC pressure for three grid resolutions.



(a) $n = 100$

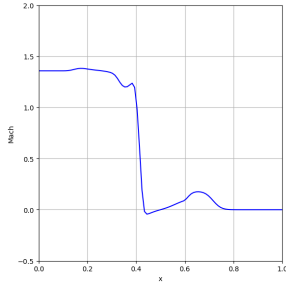


(b) $n = 200$

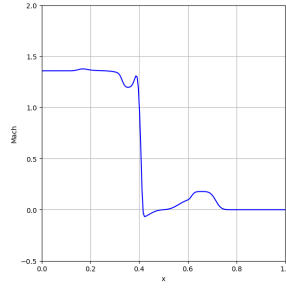


(c) $n = 400$

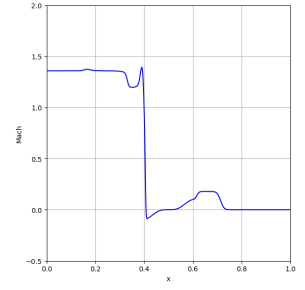
Figure 10: HLLC temperature for three grid resolutions.



(a) $n = 100$



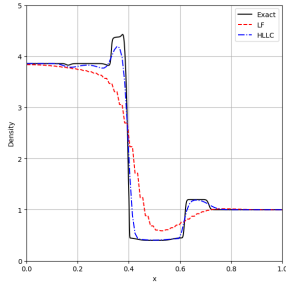
(b) $n = 200$



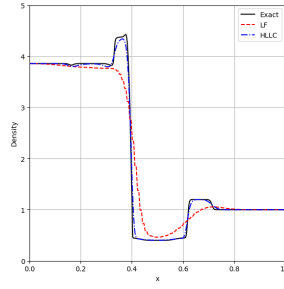
(c) $n = 400$

Figure 11: HLLC Mach number for three grid resolutions.

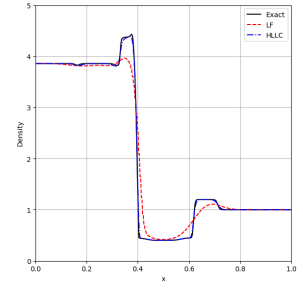
Exact Solution vs Numerical Schemes



(a) $n = 100$

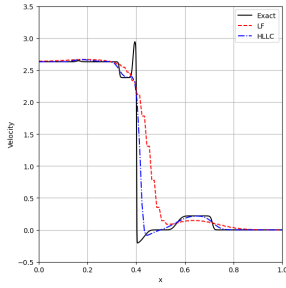


(b) $n = 200$

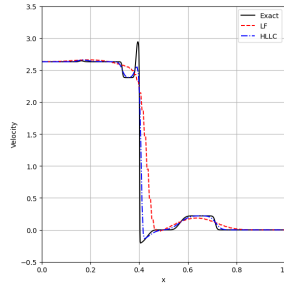


(c) $n = 400$

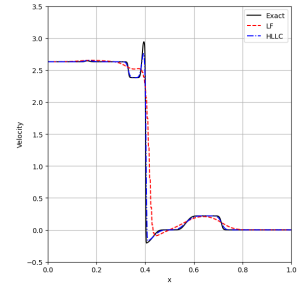
Figure 12: Comparison of density profiles (Lax–Friedrichs vs. HLLC).



(a) $n = 100$

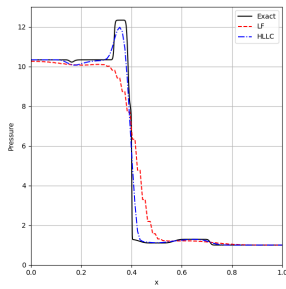


(b) $n = 200$

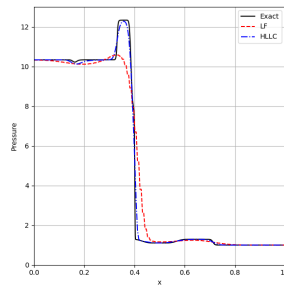


(c) $n = 400$

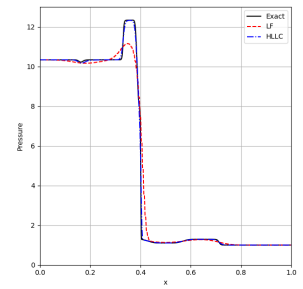
Figure 13: Comparison of velocity profiles (Lax–Friedrichs vs. HLLC).



(a) $n = 100$

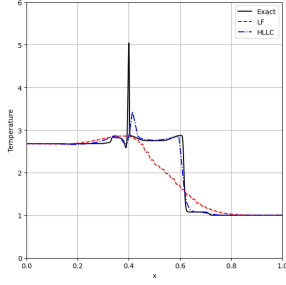


(b) $n = 200$

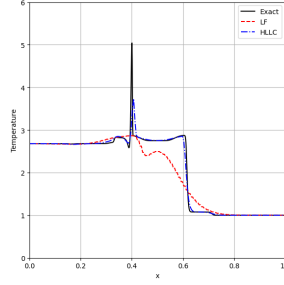


(c) $n = 400$

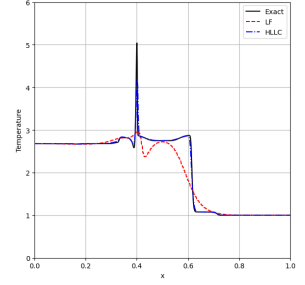
Figure 14: Comparison of pressure profiles (Lax–Friedrichs vs. HLLC).



(a) $n = 100$

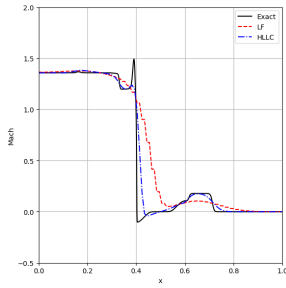


(b) $n = 200$

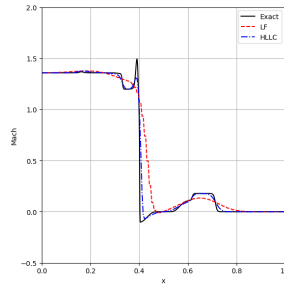


(c) $n = 400$

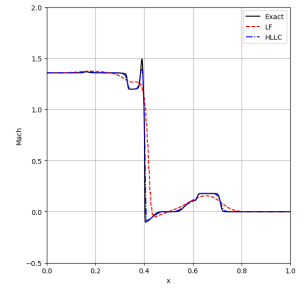
Figure 15: Comparison of temperature profiles (Lax–Friedrichs vs. HLLC)



(a) $n = 100$



(b) $n = 200$



(c) $n = 400$

Figure 16: Comparison of Mach number profiles (Lax–Friedrichs vs. HLLC)

The overlay plots confirm that Lax–Friedrichs, while stable, smears all discontinuities. In density and pressure, LF rounds off the sharp jumps and broadens the plateaus; in velocity, it replaces the steep drop at $x \approx 0.4$ with a gradual slope and blurs the small bump at $x \approx 0.65$. Temperature is most affected: LF cannot reproduce the narrow spike ($T \approx 5$) at $x \approx 0.4$, instead yielding a wide hump near $T \approx 2.5$ and a poorly resolved heated-region plateau. Its Mach profile also loses the negative dip behind the shock and underestimates the heated-region bump. In contrast, HLLC nearly overlaps the exact solution in every variable capturing the sharp density and pressure jumps, the narrow temperature peak, and the small Mach dip with only minimal smoothing. Hence, HLLC achieves substantially higher fidelity and convergence efficiency than Lax–Friedrichs.

Conclusion

This study shows that the exact self-similar solution is a rigorous benchmark for compressible flow solvers. In comparisons at $t = 0.085$, the Lax–Friedrichs method remained stable but diffused shocks and contacts unless a very fine mesh was used, leading to smeared features and small oscillations. By contrast, the HLLC solver sharply captured all discontinuities and plateau regions with minimal spurious oscillations, even on coarse grids, and converged quickly toward the exact solution as resolution increased. These results demonstrate HLLC’s clear advantage in accuracy and computational efficiency for simulating flows driven by localized thermal gradients, making it the preferred choice for practical CFD applications.

Bibliography

- [1] John D. Anderson. *Fundamentals of Aerodynamics*. McGraw-Hill Education, New York, 6th edition, 2017.
- [2] Joel H. Ferziger and Milovan Perić. *Computational Methods for Fluid Dynamics*. Springer, Berlin, Germany, 3rd edition, 2002.
- [3] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002.
- [4] Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing, New York, 1980.
- [5] Eleuterio F Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics*. Springer, 2009.

Python Code

The following code simulates the one-dimensional Euler equations with a central heated region using the Lax-Friedrich and HLLC Riemann solver.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  gamma      = 1.4
5  CFL        = 0.5
6  t_end      = 0.085
7  nx_fine    = 1000
8  nx_coarse  = 400
9
10 nx = nx_coarse
11 x  = np.linspace(0, 1, nx)
12
13 rho0, u0, p0 = 1.0, 0.0, 1.0
14 rho3, u3, p3 = 0.5, 0.0, 1.5
15
16
17 M_shock = 3.0
18 c0      = np.sqrt(gamma * p0 / rho0)
19 S12     = M_shock * c0
20 p1      = p0 * (1 + (2*gamma/(gamma+1))*(M_shock**2 - 1)
21             )
22 rho1    = rho0 * (((gamma+1)*M_shock**2) / ((gamma-1)*
```

```

    M_shock**2 + 2))
22 u1      = S12 * (1 - rho0 / rho1)
23
24
25 rho = np.ones(nx) * rho0
26 u   = np.zeros(nx)
27 p   = np.ones(nx) * p0
28
29
30 r1 = (x < 0.1)
31 r3 = (x >= 0.4) & (x <= 0.6)
32
33
34 rho[r1] = rho1; u[r1] = u1; p[r1] = p1
35 rho[r3] = rho3; u[r3] = u3; p[r3] = p3
36
37
38
39 def prim_to_cons(rho, u, p):
40     """
41     Convert primitive (rho, u, p) to conserved (rho, rho
        *u, E).
42     """
43     E = p/(gamma - 1) + 0.5 * rho * u**2
44     return np.vstack((rho, rho*u, E))
45
46 def cons_to_prim(U):
47     """
48     Convert conserved U = [rho, rho*u, E] to primitive (
        rho, u, p).
49     """
50     rho = U[0]

```



```

51     u    = U[1] / rho
52     E    = U[2]
53     p    = (gamma - 1) * (E - 0.5 * rho * u**2)
54     return rho, u, p
55
56 def euler_flux(rho, u, p):
57     """
58     Compute Euler flux  $F = [\rho u, \rho u^2 + p, (E+p)u]$ .
59     """
60     E = p/(gamma - 1) + 0.5 * rho * u**2
61     return np.array([rho*u, rho*u**2 + p, (E + p)*u])
62
63 def lax_friedrich_flux(U_L, U_R, dt, dx):
64     """
65     L a x Friedrichs numerical flux between  $U_L$  and  $U_R$ .
66     """
67     rhoL, uL, pL = cons_to_prim(U_L)
68     rhoR, uR, pR = cons_to_prim(U_R)
69     F_L = euler_flux(rhoL, uL, pL)
70     F_R = euler_flux(rhoR, uR, pR)
71     return 0.5 * (F_L + F_R) - 0.5 * (dx / dt) * (U_R -
72         U_L)
73
74 def hllc_flux(U_L, U_R):
75     """
76     HLLC numerical flux between  $U_L$  and  $U_R$ .
77     """
78     rhoL, uL, pL = cons_to_prim(U_L)
79     rhoR, uR, pR = cons_to_prim(U_R)
80     cL = np.sqrt(gamma * pL / rhoL)
81     cR = np.sqrt(gamma * pR / rhoR)

```

```

81
82     SL = min(uL - cL, uR - cR)
83     SR = max(uL + cL, uR + cR)
84
85     F_L = euler_flux(rhoL, uL, pL)
86     F_R = euler_flux(rhoR, uR, pR)
87
88     numerator    = (pR - pL + rhoL*uL*(SL - uL) - rhoR*uR
89                     *(SR - uR))
90     denominator = (rhoL*(SL - uL) - rhoR*(SR - uR))
91     S_star = numerator / denominator
92
93     def star_state(rhoK, uK, pK, S_K, S_star):
94         """
95         Compute conserved U* in the star region on side
96         K.
97         """
98         E_K = pK/(gamma - 1) + 0.5 * rhoK * uK**2
99         rho_star = rhoK * (S_K - uK) / (S_K - S_star)
100        u_star    = S_star
101        E_star = rho_star * (
102            E_K / rhoK
103            + (S_star - uK) * (S_star + pK / (rhoK * (
104                S_K - uK)))
105        )
106        return np.array([rho_star, rho_star*u_star,
107                        E_star])
108
109     U_star_L = star_state(rhoL, uL, pL, SL, S_star)
110     U_star_R = star_state(rhoR, uR, pR, SR, S_star)
111
112     if SL >= 0:

```

```

109         return F_L
110     elif SL < 0 <= S_star:
111         return F_L + SL * (U_star_L - U_L)
112     elif S_star <= 0 < SR:
113         return F_R + SR * (U_star_R - U_R)
114     else:
115         return F_R
116
117 def run_shock_heated(flux_type, nx):
118     """
119     Solve the 1D shock heated pocket problem up to
120         t_end using either 'LF' or 'HLLC'.
121     - Mach 3 shock from x=0.1
122     - Heated pocket in [0.4, 0.6]
123     Returns: (x, rho, u, p, T, Mach) at t_end.
124     """
125     x = np.linspace(0, 1, nx)
126     dx = x[1] - x[0]
127
128     rho0, u0, p0 = 1.0, 0.0, 1.0
129     rho3, u3, p3 = 0.5, 0.0, 1.5
130
131     M_shock = 3.0
132     c0 = np.sqrt(gamma * p0 / rho0)
133     S12 = M_shock * c0
134     p1 = p0 * (1 + (2*gamma/(gamma+1)) * (M_shock
135         **2 - 1))
136     rho1 = rho0 * (((gamma+1)*M_shock**2) / ((gamma
137         -1)*M_shock**2 + 2))
138     u1 = S12 * (1 - rho0 / rho1)

```

```

138     rho = np.ones(nx) * rho0
139     u    = np.zeros(nx)
140     p    = np.ones(nx) * p0
141
142
143     r1 = (x < 0.1)
144     r3 = (x >= 0.4) & (x <= 0.6)
145
146     rho[r1] = rho1; u[r1] = u1; p[r1] = p1
147     rho[r3] = rho3; u[r3] = u3; p[r3] = p3
148
149
150     U = prim_to_cons(rho, u, p)
151     t = 0.0
152
153     while t < t_end:
154         rho_n, u_n, p_n = cons_to_prim(U)
155         c_n = np.sqrt(gamma * p_n / rho_n)
156         dt = CFL * dx / np.max(np.abs(u_n) + c_n)
157         if t + dt > t_end:
158             dt = t_end - t
159
160
161         F = np.zeros((3, nx+1))
162         U_ext = np.zeros((3, nx+2))
163         U_ext[:,1:-1] = U
164         U_ext[:,0] = U[:,0]
165         U_ext[:, -1] = U[:, -1]
166
167         for i in range(nx+1):
168             if flux_type == 'LF':
169                 F[:,i] = lax_friedrich_flux(U_ext[:,i],

```

```

        U_ext[:,i+1], dt, dx)
170         else:
171             F[:,i] = hllc_flux(U_ext[:,i], U_ext[:,i
                +1])
172
173
174         for i in range(nx):
175             U[:,i] -= (dt / dx) * (F[:,i+1] - F[:,i])
176
177         t += dt
178
179         rho_f, u_f, p_f = cons_to_prim(U)
180         T_f = p_f / rho_f
181         Mach_f = u_f / np.sqrt(gamma * p_f / rho_f)
182         return x, rho_f, u_f, p_f, T_f, Mach_f
183
184
185
186
187 x_ex, rho_ex, u_ex, p_ex, temp_ex, mach_ex =
        run_shock_heated('HLLC', nx_fine)
188
189
190 plt.figure(figsize=(6, 6))
191 plt.plot(x_ex, rho_ex, 'k-')
192 plt.xlabel('x'); plt.ylabel('Density')
193 plt.xlim(0, 1); plt.ylim(0, 5)
194 plt.grid(True)
195 plt.tight_layout()
196 plt.show()
197
198

```

```

199 plt.figure(figsize=(6, 6))
200 plt.plot(x_ex, u_ex, 'k-')
201 plt.xlabel('x'); plt.ylabel('Velocity')
202 plt.xlim(0, 1); plt.ylim(-0.5, 3.5)
203 plt.grid(True)
204 plt.tight_layout()
205 plt.show()
206
207
208 plt.figure(figsize=(6, 6))
209 plt.plot(x_ex, p_ex, 'k-')
210 plt.xlabel('x'); plt.ylabel('Pressure')
211 plt.xlim(0, 1); plt.ylim(0, 13)
212 plt.grid(True)
213 plt.tight_layout()
214 plt.show()
215
216
217 plt.figure(figsize=(6, 6))
218 plt.plot(x_ex, temp_ex, 'k-')
219 plt.xlabel('x'); plt.ylabel('Temperature')
220 plt.xlim(0, 1); plt.ylim(0, 6)
221 plt.grid(True)
222 plt.tight_layout()
223 plt.show()
224
225
226 plt.figure(figsize=(6, 6))
227 plt.plot(x_ex, mach_ex, 'k-')
228 plt.xlabel('x'); plt.ylabel('Mach')
229 plt.xlim(0, 1); plt.ylim(-0.5, 2)
230 plt.grid(True)

```

```

231 plt.tight_layout()
232 plt.show()
233
234
235
236
237 x_lf, rho_lf, u_lf, p_lf, temp_lf, mach_lf =
    run_shock_heated('LF', nx_coarse)
238
239
240 plt.figure(figsize=(6, 6))
241 plt.plot(x_lf, rho_lf, 'r-')
242 plt.xlabel('x'); plt.ylabel('Density')
243 plt.xlim(0, 1); plt.ylim(0, 5)
244 plt.grid(True)
245 plt.tight_layout()
246 plt.show()
247
248
249 plt.figure(figsize=(6, 6))
250 plt.plot(x_lf, u_lf, 'r-')
251 plt.xlabel('x'); plt.ylabel('Velocity')
252 plt.xlim(0, 1); plt.ylim(-0.5, 3.5)
253 plt.grid(True)
254 plt.tight_layout()
255 plt.show()
256
257
258 plt.figure(figsize=(6, 6))
259 plt.plot(x_lf, p_lf, 'r-')
260 plt.xlabel('x'); plt.ylabel('Pressure')
261 plt.xlim(0, 1); plt.ylim(0, 13)

```

```

262 plt.grid(True)
263 plt.tight_layout()
264 plt.show()
265
266
267 plt.figure(figsize=(6, 6))
268 plt.plot(x_lf, temp_lf, 'r-')
269 plt.xlabel('x'); plt.ylabel('Temperature')
270 plt.xlim(0, 1); plt.ylim(0, 6)
271 plt.grid(True)
272 plt.tight_layout()
273 plt.show()
274
275
276 plt.figure(figsize=(6, 6))
277 plt.plot(x_lf, mach_lf, 'r-')
278 plt.xlabel('x'); plt.ylabel('Mach')
279 plt.xlim(0, 1); plt.ylim(-0.5, 2)
280 plt.grid(True)
281 plt.tight_layout()
282 plt.show()
283
284
285
286
287 x_hc, rho_hc, u_hc, p_hc, temp_hc, mach_hc =
    run_shock_heated('HLLC', nx_coarse)
288
289
290 plt.figure(figsize=(6, 6))
291 plt.plot(x_hc, rho_hc, 'b-')
292 plt.xlabel('x'); plt.ylabel('Density')

```



```

293 plt.xlim(0, 1);      plt.ylim(0, 5)
294 plt.grid(True)
295 plt.tight_layout()
296 plt.show()
297
298
299 plt.figure(figsize=(6, 6))
300 plt.plot(x_hc, u_hc, 'b-')
301 plt.xlabel('x'); plt.ylabel('Velocity')
302 plt.xlim(0, 1);      plt.ylim(-0.5, 3.5)
303 plt.grid(True)
304 plt.tight_layout()
305 plt.show()
306
307
308 plt.figure(figsize=(6, 6))
309 plt.plot(x_hc, p_hc, 'b-')
310 plt.xlabel('x'); plt.ylabel('Pressure')
311 plt.xlim(0, 1);      plt.ylim(0, 13)
312 plt.grid(True)
313 plt.tight_layout()
314 plt.show()
315
316 plt.figure(figsize=(6, 6))
317 plt.plot(x_hc, temp_hc, 'b-')
318 plt.xlabel('x'); plt.ylabel('Temperature')
319 plt.xlim(0, 1);      plt.ylim(0, 6)
320 plt.grid(True)
321 plt.tight_layout()
322 plt.show()
323
324

```

```

325 plt.figure(figsize=(6, 6))
326 plt.plot(x_hc, mach_hc, 'b-')
327 plt.xlabel('x'); plt.ylabel('Mach')
328 plt.xlim(0, 1); plt.ylim(-0.5, 2)
329 plt.grid(True)
330 plt.tight_layout()
331 plt.show()
332
333
334
335
336
337 plt.figure(figsize=(6, 6))
338 plt.plot(x_ex, rho_ex, 'k-', label='Exact')
339 plt.plot(x_lf, rho_lf, 'r--', label='LF')
340 plt.plot(x_hc, rho_hc, 'b-.', label='HLLC')
341 plt.xlabel('x'); plt.ylabel('Density')
342 plt.xlim(0, 1); plt.ylim(0, 5)
343 plt.legend(); plt.grid(True)
344 plt.tight_layout()
345 plt.show()
346
347 plt.figure(figsize=(6, 6))
348 plt.plot(x_ex, u_ex, 'k-', label='Exact')
349 plt.plot(x_lf, u_lf, 'r--', label='LF')
350 plt.plot(x_hc, u_hc, 'b-.', label='HLLC')
351 plt.xlabel('x'); plt.ylabel('Velocity')
352 plt.xlim(0, 1); plt.ylim(-0.5, 3.5)
353 plt.legend(); plt.grid(True)
354 plt.tight_layout()
355 plt.show()
356

```

```

357 plt.figure(figsize=(6, 6))
358 plt.plot(x_ex, p_ex, 'k-', label='Exact')
359 plt.plot(x_lf, p_lf, 'r--', label='LF')
360 plt.plot(x_hc, p_hc, 'b-.', label='HLLC')
361 plt.xlabel('x'); plt.ylabel('Pressure')
362 plt.xlim(0, 1); plt.ylim(0, 13)
363 plt.legend(); plt.grid(True)
364 plt.tight_layout()
365 plt.show()
366
367
368 plt.figure(figsize=(6, 6))
369 plt.plot(x_ex, temp_ex, 'k-', label='Exact')
370 plt.plot(x_lf, temp_lf, 'r--', label='LF')
371 plt.plot(x_hc, temp_hc, 'b-.', label='HLLC')
372 plt.xlabel('x'); plt.ylabel('Temperature')
373 plt.xlim(0, 1); plt.ylim(0, 6)
374 plt.legend(); plt.grid(True)
375 plt.tight_layout()
376 plt.show()
377
378 plt.figure(figsize=(6, 6))
379 plt.plot(x_ex, mach_ex, 'k-', label='Exact')
380 plt.plot(x_lf, mach_lf, 'r--', label='LF')
381 plt.plot(x_hc, mach_hc, 'b-.', label='HLLC')
382 plt.xlabel('x'); plt.ylabel('Mach')
383 plt.xlim(0, 1); plt.ylim(-0.5, 2)
384 plt.legend(); plt.grid(True)
385 plt.tight_layout()
386 plt.show()

```

Listing 1: Python implementation of Lax Friedrich and HLLC solver for the heated-region problem